

Contents

1	First-Order Logics	1
1.1	Syntax and Semantics of first-order logic	1
1.1.1	Syntax	1
1.1.2	Tensor Representation of Semantics	1
1.1.3	Worlds	2
1.1.4	Two Tensor Structures	2
1.2	Substitution Structure	3
1.2.1	Grounding Tensors	3
1.2.2	Basis encodings of terms	3
1.2.3	Atomic sentences	4
1.2.4	Formula Synthesis by Connectives	4
1.2.5	Quantifiers	5
1.2.6	Universally quantified formulas as contraction equations	7
1.2.7	Discussion	7
1.3	Inference	7
1.3.1	Manipulation of contraction equations	7
1.3.2	Unification	8
1.3.3	Modus ponens	8
1.3.4	Contraction equations	10
1.4	Representation of Knowledge Graphs	11
1.4.1	Representation as Unary and Binary Predicates	11
1.4.2	Representation as Ternary Predicate	11
1.4.3	SPARQL Queries	12
1.5	Tensor storage of Groundings	14
1.5.1	Storage in Basis CP Decomposition	14
1.5.2	Example: Relational Databases	15
	References	16

Chapter 1

First-Order Logics

Abstract To introduce the syntax and semantics of first-order logics, we in this chapter introduce the substitution tensor structure. This allows to capture structured representations of systems, beyond the factored representations introduced in the first two chapters.

1.1 Syntax and Semantics of first-order logic

The framework of first-order logic generalizes propositional logic analogously to the generalization of factored system representations towards structured system representations [6]. This more expressive framework is designed to reason about relations and functions between objects. To capture this framework by tensors we here first define the syntax and then investigate tensor representations of the semantics.

1.1.1 Syntax

A first-order logic theory consists in a finite set of constant symbols $\mathcal{U}$, a set of function symbols $\{v_l : l \in [r]\}$, a set of predicate symbols $\{g_k : k \in [d]\}$ and an arity map assigning the arity n_l to the function v_l and the arity n_k to the predicate g_k .

1.1.2 Tensor Representation of Semantics

We here follow the model-theoretic semantics of first-order logic, where sets of possible worlds are considered. We only treat in this work database semantics, where we assume that the domain $\mathcal{U}$ of each world has a one-to-one mapping to the constants of the theory and is therefore constant among the worlds. Database semantics is central to combine the semantic structure with the substitution structure. Under this assumption, the dimension of the substitution structure does not depend on the specific world, and we can combine both structures in a single tensor space to be defined in the next sections.

To each world there is a world domain $\mathcal{U}$ of objects, which we assume to be finite. We exploit the set-encoding formalism discussed in more detail in Chapter ?? and use bijective index interpretation maps

$$I : [r] \rightarrow \mathcal{U}.$$

A so-called term variable O takes states $o \in [r]$, which represent objects

$$I[o] \in \mathcal{U}.$$

The relations between objects are described by n -ary predicates g . Given a specific world x_W the truth of relations is represented by boolean tensors

$$g|_{x_W} : \bigotimes_{l \in [n]} [r] \rightarrow \{0, 1\}.$$

Given a tuple $o_0, \dots, o_{n-1} \in \bigotimes_{l \in [n]} [r]$ the boolean

$$g|_{x_W} [O_0 = o_0, \dots, O_{n-1} = o_{n-1}] \in \{0, 1\}$$

is called a grounding and encodes, whether the relation g is satisfied in the world x_W for the objects $I[o_0], \dots, I[o_{n-1}]$.

Functions in first-order logic are object-valued maps

$$v|_{x_W} : \mathcal{U}^r \rightarrow \mathcal{U}.$$

While relations are represented by their coordinate encodings, functions are represented by directed basis encodings

$$\beta^{v|_{x_W}} [O_v, O_{[n]}]$$

where to each object tuple $I[o_0], \dots, I[o_{n-1}]$ the vector $\beta^{v|_{x_W}} [O_v, O_{[n]} = o_{[n]}]$ is the one-hot encoding of the object, that is

$$\beta^{v|_{x_W}} [O_v, O_{[n]} = o_{[n]}] = \epsilon_{v|_{x_W} [I[o_0], \dots, I[o_{n-1}]]} [O_v].$$

1.1.3 Worlds

Worlds in First-Order Logics contain varying numbers of objects. We use domain variables O to enumerate the sets of objects.

Worlds are represented by three sets of tensors:

- One-hot encodings of constants
- Coordinate encodings for each predicate
- Basis encoding of each function

1.1.4 Two Tensor Structures

In comparison with propositional logic, first-order logic bears two natural tensor structures.

- **Semantic structure:** As in propositional logic, we enumerate possible worlds by a collection of variables X . This is a representation of the model-theoretic semantics, where subsets of worlds are represented by boolean tensors.
- **Substitution structure:** Different to propositional logic, a formula can have variables and the evaluation of a formula on a world is represented by its grounding tensor. Given a world which contains objects in the domain $\mathcal{U}$, we can substitute variables by objects in the domain. In this way, each predicate with r variables is represented in that world as a boolean tensor of order r .

While the semantic structure appears already in factored representations of systems, the substitution structure arises in more generality when treating structured representations of systems [6].

1.2 Substitution Structure

We here investigate the structure of worlds in first-order logics and how they can be represented by tensors. As we already argued, groundings of sentences can be aligned in boolean tensors, to be called grounding tensors. We here investigate sparse representation schemes for this substitution structure. In particular we apply basis calculus to find sparse decompositions of the grounding tensors

The semantic structure, an attempt to represent in tensors multiple worlds at once, is to be investigated in Chapter ??.

1.2.1 Grounding Tensors

Given a first-order logic world x_W , arbitrary formulas are interpreted in terms of the satisfactions of their groundings. We define their semantic first, and then relate their syntactical decomposition to tensor networks, similar to our approach to propositional logic in Chapter ??.

Definition 1.1 (Grounding of a first-order formula given a world) Given a specific world x_W , with a domain $\mathcal{U}$ enumerated by $[r]$, the grounding of a formula q with variables O_q is the tensor

$$q|_{x_W} [O_q] : \bigotimes_{l \in [O_q]} [r] \rightarrow \{0, 1\}.$$

Each coordinate represents thereby the boolean, whether the substitution of the variables in the formula is satisfied given a world x_W , that is

$$q|_{x_W} [O_q = o_q] = 1$$

if and only if the substitution of q with the variables O_q replaced by the objects $I[o_l]$ is satisfied on the world x_W .

We in the following investigate how these tensors are represented by tensor networks involving the world defining tensors.

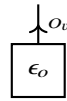
1.2.2 Basis encodings of terms

Terms are object valued functions, which r_v different arguments are also object valued and called variables. We represent terms by the corresponding basis encodings

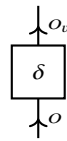
$$\beta^v [O_v, O_{[r_v]}]$$

Basic blocks:

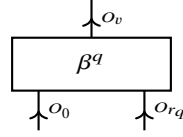
- Constant



- Variable

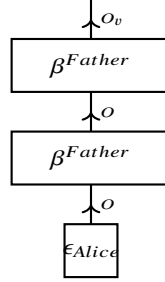


- Function of terms



Contractions of those terms along directed acyclic graphs with single root nodes represent more complex terms. As such, basis encodings of functions can be contracted with constants, variables or other functions to build up basis encodings of more complex terms.

Example: Father of the father of alice is represented by the contraction

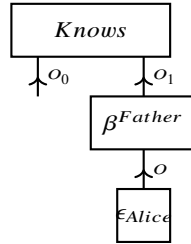


We notice, that the basis encoding of each term is decomposed into a network of world defining tensors, namely constant and function defining tensors.

1.2.3 Atomic sentences

Atomic sentences are contractions of coordinate encodings of predicates with basis encodings of terms.

Example: Knowing the father of alice



Complex sentences are built by

- Quantifier free sentences are contractions of basis encodings to atomic sentences with basis encodings of the connectives.
- Quantifiers act as non-linear transforms of tensors.

1.2.4 Formula Synthesis by Connectives

In order to have a sound semantic, the grounding of first-order logic formulas is determined by the syntax of the formula, i.e. a decomposition of the formula into connectives and quantifiers acting on atomic formulas.

Quantifier-free formulas are connectives acting on atomic formulas. We can describe them as in the case of propositional logic in the β -formalism. While the atomic formulas where delta tensors copying states, they are more involved here.

Theorem 1.1 *For any connective $\circ$ and formulas q_1 and q_2 we have*

$$(q_1 \circ q_2)|_{x_W} [O_{q_1} \cup O_{q_2}] \quad (1.1)$$

$$= \langle \beta^{q_1|_{x_W}} [Y_{q_1}, O_{q_1}], \beta^{q_2|_{x_W}} [Y_{q_2}, O_{q_2}], \beta^\circ [Y_{q_1 \circ q_2}, Y_{q_1}, Y_{q_2}], \epsilon_1 [Y_{q_1 \circ q_2}] \rangle_{[O_{[n]}]} . \quad (1.2)$$

Proof. This directly follows from Thm. ??.

□

Here, variables can be shared by the connected formulas, therefore the variables in the combined formula are unions of the possible not disjoint variables of the connected formulas.

When interpreting the head variables of relational encoded atomic formulas as the atoms of a propositional theory, we find a propositional formula f associated with any decomposable first-order logic formula.

Definition 1.2 Given a formula q in first-order logic, we say that a propositional formula $f [X_{[d]}]$ is the propositional equivalent to q given atomic formulas g_k in first-order logic, when for any world x_W we have

$$q|_{x_W} [O_q] = \langle \{ \beta^{g_k|_{x_W}} [X_k, O_{g_k}] : k \in [d] \} \cup \{ f [X_{[d]}] \} \rangle_{[O_q]} .$$

We here denote the head variables of the basis encoding to $\beta^{g_k|_{x_W}}$ by X_k to highlight their interpretation as propositional atoms.

We depict the relation of a grounding tensor to a propositional formula in Figure 1.1.

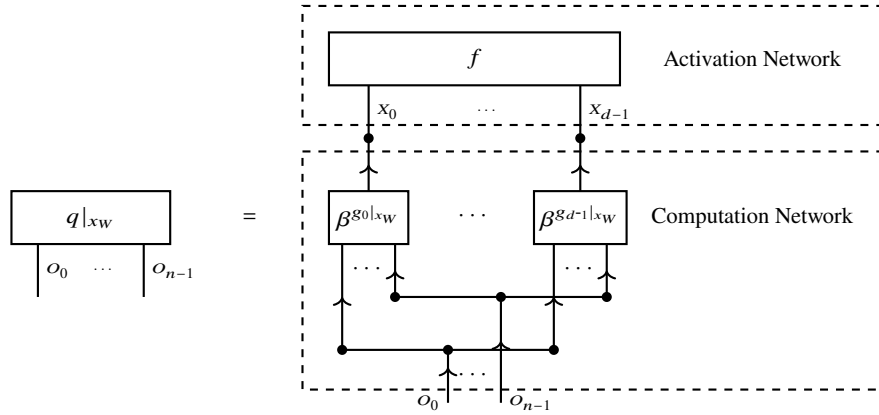


Fig. 1.1 Grounding of a quantifier-free first-order logic formula q is a Computation-Activation Network: The atomic formulas g_k are computed and activated by a propositional formula $f [X_{[d]}]$. The computation network therefore consists of the of the (possibly substituted) predicate groundings, while the activation network consists of the (possibly decomposed) propositional formula $f [X_{[d]}]$.

1.2.5 Quantifiers

Existential and universal quantifiers appear in generic first-order logic and are besides substitutions further means to reduce the number of variables in a formula.

The semantics of existential quantification consists in a formula being true, if at least one state of the quantified variable is true, as we define next.

Definition 1.3 Given a grounding tensor

$$q|_{x_W} [O_0, \dots, O_{n-1}]$$

the existential and universal quantification with respect to the first variable are the tensors

$$(\exists_{o_0} q)|_{x_W} [O_1, \dots, O_{n-1}] \quad \text{and} \quad (\forall_{o_0} q)|_{x_W} [O_1, \dots, O_{n-1}]$$

with coordinates as follows. For an assignment $o_1, \dots, o$ to the non-quantified variables we have

$$(\exists o_0 q)|_{x_W} [O_1 = o_1, \dots, O_{n-1} = o_{n-1}] = 1$$

if and only if there is an assignment $o_0 \in [r_0]$ such that

$$q|_{x_W} [O_0 = o_0, O_1 = o_1, \dots, O_{n-1} = o_{n-1}] = 1.$$

Conversely, we have for the universal quantification that

$$(\forall o_0 q)|_{x_W} [O_1 = o_1, \dots, O_{n-1} = o_{n-1}] = 1$$

if and only if for any assignment $o_0 \in [r_0]$ we have

$$q|_{x_W} [O_0 = o_0, O_1 = o_1, \dots, O_{n-1} = o_{n-1}] = 1.$$

Let us now show, that existential and universal quantification are coordinatewise transforms (see Def. ??) of contracted grounding tensors. To this end, let us introduce the greater- z indicator $\mathbb{I}_{>z}$, where $z \in \mathbb{R}$, as the function

$$\mathbb{I}_{>z} : \mathbb{R} \rightarrow \{0, 1\} \quad , \quad \mathbb{I}_{>z}(x) = \begin{cases} 1 & \text{if } x > z \\ 0 & \text{else} \end{cases}.$$

Theorem 1.2 *For any formula q with variables $O_{[n]}$ we have*

$$(\exists o_0 q)|_{x_W} [O_1, \dots, O_{n-1}] = \mathbb{I}_{>0} \left(\langle q|_{x_W} \rangle_{[O_1, \dots, O_{n-1}]} \right) [O_1, \dots, O_{n-1}]$$

and

$$(\forall o_0 q)|_{x_W} [O_1, \dots, O_{n-1}] = \mathbb{I}_{>r-1} \left(\langle q|_{x_W} \rangle_{[O_1, \dots, O_{n-1}]} \right) [O_1, \dots, O_{n-1}]$$

Proof. We proof the claimed equalities to arbitrary slices of the remaining variables, which amount to arbitrary substitutions of the formulas. For any indices $o_1 \in [r_1], \dots, o_{n-1} \in [r_{n-1}]$ We notice that

$$\begin{aligned} \langle q|_{x_W} \rangle_{[O_1=o_1, \dots, O_{n-1}=o_{n-1}]} &= \sum_{o_0 \in [r_0]} q|_{x_W} [O_0 = o_0, \dots, O_{n-1} = o_{n-1}] \\ &= |\{o_0 \in [r_0] : q|_{x_W} [O_0 = o_0, \dots, O_{n-1} = o_{n-1}] = 1\}|. \end{aligned}$$

We can thus understand the contracted grounding tensor as storing in its coordinates the count of the coordinate extensions to the zeroth variable, such that the grounding tensor is satisfied. This is analogous to our interpretation of contracted propositional formulas as world counts. From this it is obvious, that the existential quantification is satisfied, if the count is different from zero, which is captured by the coordinatewise transform with $\mathbb{I}_{>0}$. We therefore arrive at

$$(\exists o_0 q)|_{x_W} [O_1 = o_1, \dots, O_{n-1} = o_{n-1}] = \mathbb{I}_{>0} \left(\langle q|_{x_W} \rangle_{[O_1, \dots, O_{n-1}]} \right) [O_1 = o_1, \dots, O_{n-1} = o_{n-1}].$$

The first claim follows, since the assignment to the non-quantified variables was arbitrary. The universal quantification is satisfied, when all extensions are satisfied, and the count is r . Since r is the maximal count, this is captured by the coordinatewise transform with $\mathbb{I}_{>r-1}$ and we get

$$(\forall o_0 q)|_{x_W} [O_1 = o_1, \dots, O_{n-1} = o_{n-1}] = \mathbb{I}_{>r-1} \left(\langle q|_{x_W} \rangle_{[O_1, \dots, O_{n-1}]} \right) [O_1 = o_1, \dots, O_{n-1} = o_{n-1}].$$

With the same argument, the second claim is established. $\square$

1.2.6 Universally quantified formulas as contraction equations

When restricting to universally quantified formulas, we can understand them as contraction equations

FOL knowledge bases without existential quantifiers are understood as contraction equations. That is, exactly those worlds are models, for which

$$\left\langle \{q_\ell [O_{[r_{q_\ell}]}] : \ell \in [p]\} \right\rangle_{[O_V]} = \mathbb{I}[O_V] .$$

1.2.7 Discussion

We can extend this discussion towards more generic counting quantifiers, of which the existential and the universal quantifier are extreme cases. One can define quantifiers by demanding that at least $z \in \mathbb{N}$ compatible groundings are satisfied, and show that they amount to coordinatewise transforms with $\mathbb{I}_{>z}$. What is more, quantifiers demanding that at most $z \in \mathbb{N}$ are satisfied would be representable by transforms with an analogously defined function $\mathbb{I}_{\leq z}$. Such customized quantifiers appear for example in the OWL 2 standard of description logics (see [5] and Sect. 1.4).

As will be discussed in Chapter ??, any coordinatewise transform can be performed by a contraction of a basis encoding of the tensor with a head vector prepared by the transform function (see Thm. ??). In the case here, a direct implementation would require a dimension of these head variables by r , which can be infeasible when having large object sets.

To summarize, let us assume a formula is in its prenex normal form, that is a collection of quantifiers are acting on a quantifier free part. We can represent its grounding tensor in three steps:

- Instantiations of the atom groundings with the assigned variables, as contractions of the basis encoding of the world tensor with atom selecting tensors.
- Propositional formula acting on the head variables of the predicate instantiations, representing the connectives combining the formula.
- Quantifiers as a composition of contractions closing the quantified variable and coordinatewise transforms with the respective greater-than indicators.

1.3 Inference

Substitutions are used to lift inference to propositional logics:

- Represent the formulas to be inferred with the same computation network of atomic formulas.
- Represent the formulas with an entangled activation network (whereas before they were disentangled, since referring to different atomic sentences).

1.3.1 Manipulation of contraction equations

Universally quantified FOL formulas are equations of FOL Computation-Activation Networks to the trivial tensor.

We understand inference as manipulations in the activation network.

- **Substituion:** Substitution of variables to represent multiple sentences by same activation network. The task of unification is to find a substitution, such that two computation networks can be merged with same term variables.
- **Propositional inference:** Manipulations in activation network, as for constraint networks
- **Restrictions to parts:** Any sub-network of the activation network in a contraction equation poses an implied contraction equation.

1.3.1.1 Substitution

Substitution is the contraction with one-hot encodings to the term variables. It is easy to see, that when a contraction equation holds, it also holds for any substitution of the term variables. Note that substitution changes the computation network, but not the activation network.

1.3.1.2 Propositional inference

We understand propositional inference as the simplication of the activation network. Whenever a propositional formula is inferred by the activation network, we can add it to the activation network as a new decorated edge.

1.3.1.3 Extraction of entailed formulas

We can extract entailed formulas from a contraction equation by restricting the activation network to a sub-network, as we show as the next theorem. Therefore any result of the propositional inference, in the shape of a Boolean tensor added to the activation network, can be used to extract entailed formulas.

Theorem 1.3 *Let $\tau^{\mathcal{G}}$ be an activation network of boolean tensors on a hypergraph $\mathcal{G}$ and let $\tilde{\mathcal{E}} \subset \mathcal{E}$. Let $g_{[d]}$ be an arbitrary set of first order predicates with the term variables I_f*

$$\langle \tau^{\mathcal{G}} \cup \{\beta^k [X_k, O_{[n]}] : k \in [d]\} \rangle_{[O_{[n]}]} = \mathbb{I} [X_{[n]}]$$

then also

$$\langle \{\tau^e [X_e] : e \in \tilde{\mathcal{E}}\} \cup \{\beta^k [X_k, O_{[n]}] : k \in [d]\} \rangle_{[O_{[n]}]} = \mathbb{I} [X_{[n]}]$$

Proof. Notice, that the contraction with the smaller set of edges $\tilde{\mathcal{E}} \subset \mathcal{E}$ is a Boolean tensor. By the monotonicity of the contraction, we have that the support of the contraction is smaller (in the partial order of tensors) to that of all edges. Since by assumption the latter is the ones tensors, the former must also be the ones tensor. $\square$

1.3.2 Unification

Unification of atomic sentences: Find a substitution to the variables, such that both sentences are represented by the same computation network.

Compressing of complex sentences:

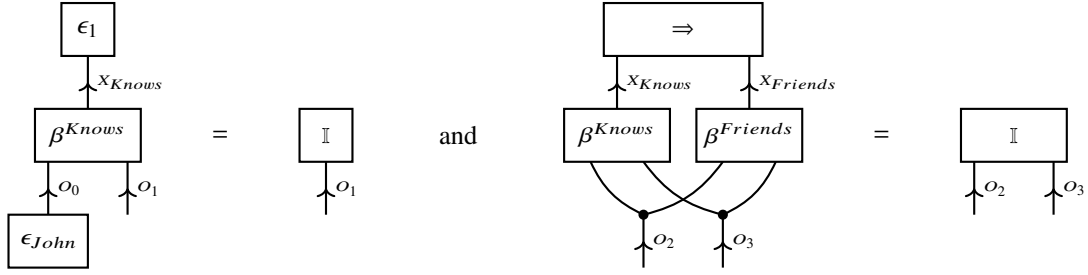
- Find a unification of the atomic sentences in both complex sentences
- Represent the substituted complex sentences by a constraction of the activation networks

1.3.3 Modus ponens

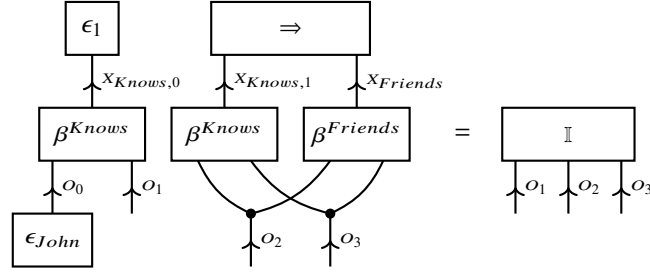
Example 1.1 (Lifted modus ponens) For example we have

- John knows everyone: $\text{Knows}(\text{John}, z)$
- People who know each other are friends: $\text{Knows}(x, y) \Rightarrow \text{Friends}(x, y)$

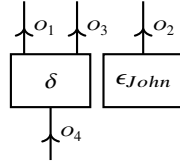
We illustrate the derivation of the entailed formula $\text{Friends}(\text{John}, z)$ (i.e. John is a friend of everyone) by modus ponens. The formulas are represented by the contraction equation



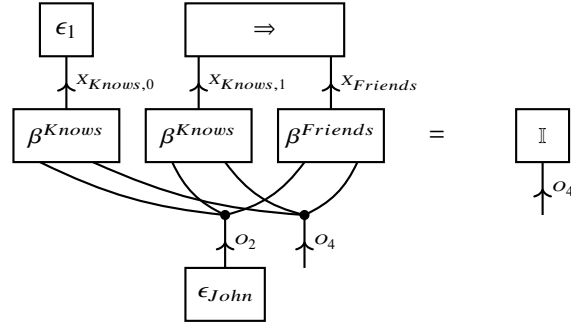
We combine them by the tensor product of both sides as



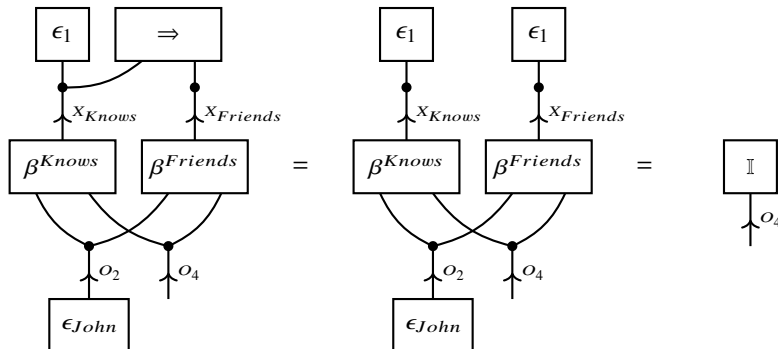
The necessary substitution for the unification of both is $O_2 \mapsto \text{John}$ and $O_1, O_3 \mapsto O_4$. This amounts to contraction of both sides with the tensor



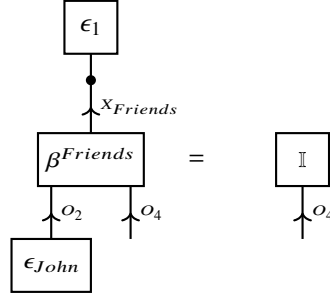
We contract this tensor to both sides of the combined contraction equation and get



Now we notice that the computation network of the variables $X_{Knows,0}$ and $X_{Knows,1}$ are equivalent (what was the goal of the unification). We simplify the equation to



Here we applied modus ponens on the activation network in the first equation. Restricting the activation network to the variable $X_{Friends}$ is then the derived formula $\text{Friends}(\text{John}, z)$ (i.e. John is a friend of everyone):



1.3.4 Contraction equations

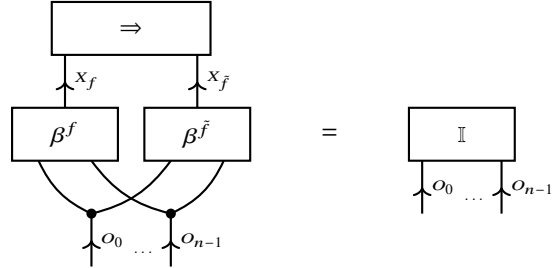
Quantified formulas understood are contraction equations.

Understanding them as hard constraints, one can infer parts of the substitution tensors, which holds for all supported worlds.

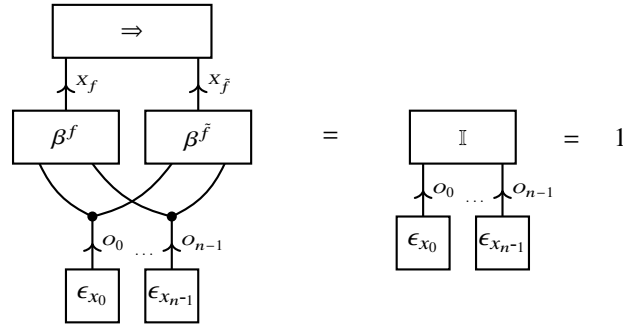
Example 1.2 (Modus ponens on the substitution structure) We here exemplify an application of unification to apply the modus ponens inference rule in the substitution structure. Let us consider the formula

$$\forall x_{[d]} : f[X_{[d]} = x_{[d]}] \Rightarrow \tilde{f}[X_{[d]} = x_{[d]}],$$

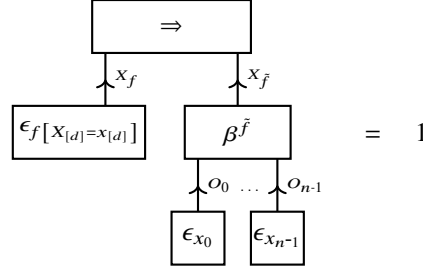
which corresponds to the contraction equation



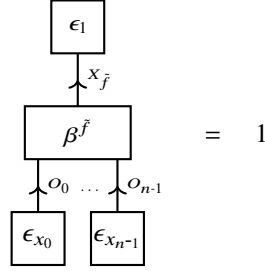
When knowing for some $x_{[d]} \in \times_{k \in [d]} [m_k]$ that $f[X_{[d]} = x_{[d]}] = 1$, this is the contraction equation we conclude that $\tilde{f}[X_{[d]} = x_{[d]}] = 1$.



We simplify the left hand side using basis calculus to



When known that $f[X_{[d]} = x_{[d]}] = 1$, this further simplifies to



which is equal to $\tilde{f}[X_{[d]} = x_{[d]}] = 1$.

More sophisticated reasoning schemes based on contraction equations are applied in [7, 8], where recursive linear equations are derived.

1.4 Representation of Knowledge Graphs

Knowledge graphs as a specific fragment of FOL. Here we focus on A-Box representation by sparse tensors.

Let us now represent a specific fragment of first-order logic, namely Description Logics which Knowledge Bases are often referred to as Knowledge Graphs. We here use the OWL 2 standard, which encodes the syntax of the description logic *SRQI*($\mathcal{D}$) [5]. It is further applied as a foundation of semantic web [1]. Knowledge bases in this fragment are further called Knowledge Graphs [3], due to a graphical interpretation. Note that the relation to tensor representations is here through extended adjacency matrices, and not decorations of the graphs by tensors as in the formalism here.

1.4.1 Representation as Unary and Binary Predicates

Predicates in knowledge graphs are binary (owl:ObjectProperties) and unary (owl:Class). We enumerate the predicates by $[d]$, the objects in the domain $\mathcal{U}$ by $[r]$, and extend the unary predicates to binaries by tensor product with $\epsilon_0 [O_1]$. A Knowledge Graph on the set $\mathcal{U}$ of constants (owl:NamedIndividuals) is then the tensor

$$\text{KG}|_{x_w} [L, O_0, O_1] : [d] \times [r] \times [r] \rightarrow \{0, 1\}.$$

1.4.2 Representation as Ternary Predicate

It has been particularly convenient to represent a Knowledge Graph instead as a grounding of a single ternary predicate RDF. To this end, the predicates g_k and another object `rdftype` are added to a domain $\mathcal{U}$, by extending the r and the index interpretation function accordingly.

Following our notation we understand a Knowledge Graph as a grounding of the `rdf` triple relation RDF (being a formula of order 3) on a specific world $\text{KG}|_{x_w}$ with individuals $\mathcal{U}$

We then construct a grounding tensor $\text{RDF}|_{x_W}$ out of the world $\text{KG}|_{x_W} [L, O_0, O_1]$ by

$$\text{RDF}|_{x_W} : [r] \times [r] \times [r] \rightarrow \{0, 1\}$$

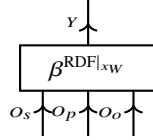
where

$$\begin{aligned} \text{RDF}|_{x_W} [O_s = o_s, O_p = o_p, O_o = o_o] \\ = \begin{cases} \text{KG}|_{x_W} [L = o_s, O_0 = o_o, O_1 = 0] & \text{if } o_p = I^{-1}(\text{rdftype}) \\ \text{KG}|_{x_W} [L = o_p, O_0 = o_s, O_1 = o_o] & \text{if } o_p = I^{-1}(g_k) \text{ for some } k \\ 0 & \text{else} \end{cases} \end{aligned}$$

Slicing the tensor $\text{RDF}|_{x_W}$ along the predicate axis retrieves specific information about roles and can be efficiently be performed on these formats. The role `rdftype` has a specific meaning, since it contains from a DL perspective classifications (memberships of named concepts). Further slicing the tensor along object axis therefore results in membership lists for specific classes (concepts). One can thus regard `rdftype` as a placeholder for unitary formulas in a space of binary formulas.

Exploiting the ℓ_0 -sparsity now leads to a so-called triple store, where $\text{RDF}|_{x_W}$ is stored by a listing of those triples o_s, o_p, o_o such that $\text{RDF}|_{x_W} [O_s = o_s, O_p = o_p, O_o = o_o] = 1$. A recent implementation of a triple store exploiting these intuitions is TETRIS, see [2]. In this work, such decompositions are generalized into more generic CP formats, see Chapter ?? . Approximations of grounding tensors by decompositions leads to embeddings of the individuals such as Tucker, ComplEx and RESCAL (see [4]).

For our purposes of evaluating logical formulas such as SPARQL queries we use the basis encoding of the groundings, which are depicted by



1.4.3 SPARQL Queries

The SPARQL query language is a syntax to express first-order logic formulas q and intended to be evaluated given a Knowledge Graph. We here consider tensor network representations of the WHERE block. Given a specific knowledge graph $\text{RDF}|_{x_W}$, the execution of query is the interpretation $q|_{x_W}$, typically represented in a sparse basis CP format where each slice represents a solution mapping.

1.4.3.1 Triple Patterns

Central to SPARQL queries are triple patterns, which we understand as slicings of the tensor $\text{RDF}|_{x_W}$. To each so-called triple pattern we build a corresponding creation tensor. The triple pattern is then evaluated by contraction of the atom creating tensor with $\text{RDF}|_{x_W}$.

Let us now provide examples of such pattern tensors. A unary triple patterns contains a single projection variable, typically related with the subject variable O_s of $\text{RDF}|_{x_W}$. The corresponding pattern tensor is then

$$\psi_{\langle Z, \text{rdftype}, g_k \rangle} [O_s, O_p, O_o, Z] = \delta [O_s, Z] \otimes \epsilon_{I^{-1}(\text{rdftype})} [O_p] \otimes \epsilon_{I^{-1}(g_k)} [O_o] .$$

Binary triple patterns come with two projection variables, typically related with the subject and the object variables O_s and O_o . The pattern tensor to the k -th predicate is then

$$\psi_{\langle Z_0, g_k, Z_1 \rangle} [O_s, O_p, O_o, Z_0, Z_1] = \delta [O_s, Z_0] \otimes \epsilon_{I^{-1}(g_k)} [O_p] \otimes \delta [O_o, Z_1] .$$

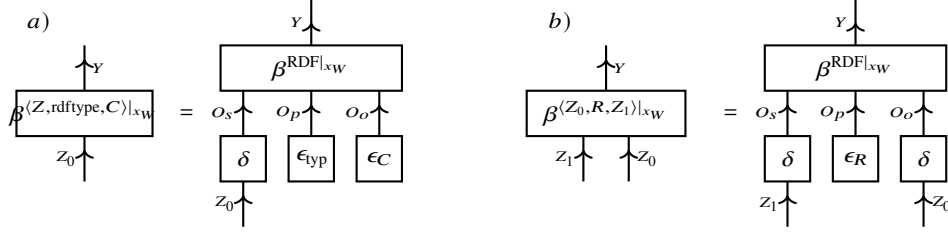


Fig. 1.2 Triple patterns of SPARQL as tensor networks. a) Example of unary triple pattern $\langle Z, \text{rdf:type}, C \rangle$ specifying whether an individual I_{o_1} is a member of class C . b) Example of a binary triple pattern $\langle Z_0, R, Z_1 \rangle$ specifying whether individuals I_{o_1} and I_{o_2} have a relation R . By ϵ_{typ} , ϵ_C , ϵ_R we denote the one-hot encodings of the enumeration of the resources rdf:type , C and R .

Contraction with these pattern tensor evaluated the specific triple pattern, and outputs in a boolean tensor the indicator, which objects are members of a specific class (for unary patterns) or which pair of objects are related by a specific relation. Again, the output of such contractions is a subset encodings of the set of solutions (see Def. ??).

Examples of triple patterns, drawn in Figure 1.2 are

- Unary triple pattern with one variable, representing a formula with a single projection variable. For the example $\langle Z, \text{rdf:type}, C \rangle$ see Figure 1.2a.

$$\psi_{\langle Z, \text{rdf:type}, g_k \rangle} [O_s, O_p, O_o, Z] = \delta [O_s, Z] \otimes \epsilon_{I^{-1}(\text{rdf:type})} [O_p] \otimes \epsilon_{I^{-1}(C)} [O_o]$$

If and only if the output slice is ϵ_1 , then the corresponding object encoded by the input indices is of class C .

- Binary triple pattern with two variables, representing a formula with two projection variables. For the example $\langle Z_0, R, Z_1 \rangle$ see Figure 1.2b. If and only if the output slice is ϵ_1 , then the corresponding object tuple encoded by the input indices has a relation R .

The composition $\psi(\psi^T)$ of the matrification of the tensor ψ is an orthogonal projection. That means that applying $\psi(\psi^T)$ is the same map as applying once.

1.4.3.2 Basic Graph Patterns

Generic SPARQL queries are compositions of triple patterns by logical connectives. These triple patterns possibly share projection variables. Statements in SPARQL can be translated into propositional logic combining the triple patterns:

SPARQL	Propositional logic	Tensor Representation
$\{f_1, f_2\}$	$f_1 \wedge f_2$	$\beta^\wedge [Y_{f_1 \wedge f_2}, Y_{f_1}, Y_{f_2}]$
UNION $\{f_1, f_2\}$	$f_1 \vee f_2$	$\beta^\vee [Y_{f_1 \vee f_2}, Y_{f_1}, Y_{f_2}]$
FILTER NOT EXISTS $\{f\}$	$\neg f$	$\beta^\neg [Y_{\neg f}, Y_f]$

If a SPARQL query consists of these keywords, we find a straight forward corresponding network of triple patterns and encoded logical connectives, by applying our findings of Sect. 1.2.4. To this end, we prepare for each appearing triple pattern the corresponding pattern tensor, and a copy of $\text{RDF}|_{x_W}$. Here we also copy the term variables O_s, O_p and O_o , to ensure that each copy of $\text{RDF}|_{x_W}$ shares variables with a single pattern tensor. Projection variables are not copied, since we need to keep track of them shared among triple patterns. Then we prepare the basis encoding of logical connectives according to the hierarchy specified in the SPARQL query. Finally we add a ϵ_1 -vector to the final head variable representing the complete SPARQL query, to restrict the support to coordinates corresponding with solution mappings. We then contract the resulting tensor network, leaving all projection variables open.

If a projection variable is not appearing in the SELECT statement in front of the WHERE $\{\cdot\}$ -block, we simply exclude it from the open variables of the described contraction. Note that in that case, the coordinates contain solution counts, i.e. how many assignments to the dropped variable have been a 1 coordinate. We can drop this additional information simply by performing a coordinatewise transform with the greater zero indicator $\mathbb{I}_{>0}$.

Here we represented a SPARQL query p consistent of multiple triple pattern by instantiating a head variables to each triple pattern. Alternatively, the more direct hybrid calculus developed in Sect. ?? can be applied and the additional

head variables avoided. This is especially compelling, when the WHERE{·}-block does not contain further keywords, i.e. it is the conjunction of all triple patterns. In that case, we avoid the instantiation of head variables (i.e. close the head variables separately by ϵ_1 -vectors) and represent the query by a contraction of all triple pattern tensors.

We further notice, that any propositional formula acting on the head variables of the triple patterns can be expressed by a hierarchical combination of the key words in the above table. To find the expression, one can transform a given formula into its conjunctive or disjunctive normal form and apply the statements according to the appearing operations $\wedge$, $\vee$ and $\neg$.

1.5 Tensor storage of Groundings

A typical storage of a grounding is the enlistment of all non-zero coordinates. We understand this as a basis CP decomposition of the grounding tensor. We then propose more efficient storage tensor formats, which are respecting the structure of the grounded formula.

1.5.1 Storage in Basis CP Decomposition

In many situations, grounding cores are sparse and representations as single tensor cores comes with a drastic overhead. We often encounter sparse grounding tensors, where the number of non-zero coordinates (to be investigated by basis CP ranks in Chapter ??) satisfies

$$\ell_0(q|_{x_W}) \ll r|O_q|.$$

In this case, since most coordinates vanish, the basis CP decomposition (see Sect. ??) enables a representation of the grounding with significantly lower storage demand, see Thm. ??. This is particularly useful for representing large relational databases, where each object has only a few relations with others, while the majority of possible relations remains unsatisfied. We depict such CP decomposition of a formula grounding in Thm. 1.3.

Most logical syntaxes exploit ℓ_0 -sparsity, explicitly storing only known assertions. The interpretation of unspecified assertions depends on the underlying assumptions. Under the Closed World Assumption, for example, all unspecified assertions are assumed to be false.

Example 1.3 Consider the binary relation *Knows* on the set of individuals $\{Alice, Bob, Charlie\}$ defined by the following pairs:

Individual 0	Individual 1
Alice	Bob
Bob	Alice
Charlie	Bob
Bob	Charlie

$$Knows[O_0, O_1] = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} [O_0, O_1].$$

with a CP representation with the leg cores

$$\rho^0 [O_0, I] = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} [O_0, I] \quad \text{and} \quad \rho^1 [O_1, I] = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} [O_1, I].$$

Consider now an extension

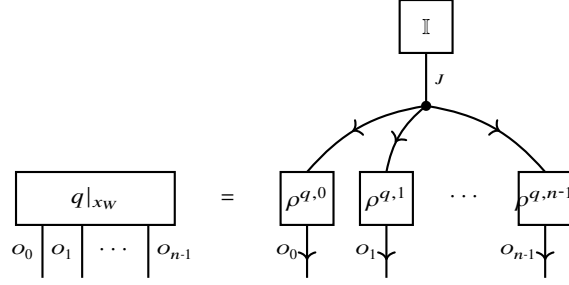


Fig. 1.3 Basis CP Decomposition of the grounding of q , following the scheme of Thm. ???. Instead of direct storage of the grounding tensor $q|_{x_W}$, the non-zero coordinates are enumerated by a variable J and the corresponding coordinates stored in leg-matrices $\rho^{q,l}$.

Individual 0	Individual 1
Alice	Bob
Bob	Alice
Charlie	Bob
Bob	Charlie
John	Alice
John	Bob
John	John
Alice	John
Bob	John
Charlie	John

Enumerating them in the order $[Alice, Bob, Charlie, John]$ this is the matrix

$$Knows[O_0, O_1] = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} [O_0, O_1] .$$

This has basis rank of 11. Notice, that allowing for bas+ representation, we would reduce to 5, by encoding that John knows everyone, and everyone knows John.

$$\rho^0 [O_0, I] = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix} [O_0, I] \quad \text{and} \quad \rho^1 [O_1, I] = \begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix} [O_1, I] .$$

1.5.2 Example: Relational Databases

A database is understood as a specific first-order logic world, and are operations on such a single world. Queries are described by a formula p , which are asked against a specific world x_W to retrieve the grounding $p|_{x_W}$. The variables of such formulas are called projection variables. The answer $p|_{x_W}$ of a query is most conveniently represented as a list of solution mappings from the projection variables to objects in the world, such that the query formula is satisfied. Answering a query by solution mappings corresponds with finding the basis CP Decomposition (see Sect. ??) of $p|_{x_W}$. We can understand these solution mappings as stored in the leg-matrices $\rho^{q,l}$ (see Figure 1.3).

Let us give with the outer join an example of a popular operation to define queries, which efficient execution and storage can be improved based on considerations in the tensor network formalism.

Definition 1.4 (Outer join) Let there be a world x_W and formulas g_ℓ depending on variables $O_{\mathcal{V}^\ell}$, which have grounding tensors by

$$g_\ell|_{x_W} [O_v] : \bigotimes_{v \in \mathcal{V}^\ell} [r_v] \rightarrow \{0, 1\}.$$

Then their (outer) JOIN is defined as the grounding of their conjunctions, as

$$\text{JOIN}(g_0, \dots, g_{p-1})|_{x_W} \left[\bigcup_{\ell \in [p]} O_{\mathcal{V}^\ell} \right] = \langle g_\ell|_{x_W} [O_{\mathcal{V}^\ell}] : \ell \in [p] \rangle_{[\bigcup_{\ell \in [p]} O_{\mathcal{V}^\ell}]}.$$

We can understand the JOIN of groundings by a factor graph, where each grounding tensor decorates the hyperedge to the node set $\mathcal{V}^\ell$. The projection variable assignment to each formula combined in a JOIN operation provide a basic tensor network format to store the output of the operation. There are thus situations, in which the solution map storage corresponding with a CP Decomposition comes with unnecessary overheads compared with other formats.

We can also understand the JOIN operation as a coordinatewise transform (see Def. ??) with the product as transform function. To make this connection solid, one would need to extend each joined formula trivially to the variables appearing in other formulas.

The efficiency of evaluating the contraction to a JOIN operation might be improved by understanding it as an Constraint Satisfaction Problem (see Chapter ??). When applying efficient Message Passing algorithms such as Knowledge Propagation (see Algorithm ??), the groundings can be sparsified by local constraint propagation operations before turning to more global and more demanding contraction operations. Here the groundings $g_\ell|_{x_W}$ would be used to initialize Knowledge Cores κ^e and sequentially sparsified during the algorithm.

References

1. Grigoris Antoniou, Paul Groth, Frank Van Harmelen, and Rinke Hoekstra. *A Semantic Web Primer, third edition*. The MIT Press, Cambridge (Mass.), third edition edition, August 2012.
2. Alexander Bigerl, Felix Conrads, Charlotte Behning, Mohamed Ahmed Sherif, Muhammad Saleem, and Axel-Cyrille Ngonga Ngomo. Tentriss – A Tensor-Based Triple Store. In Jeff Z. Pan, Valentina Tamma, Claudia d’Amato, Krzysztof Janowicz, Bo Fu, Axel Polleres, Oshani Seneviratne, and Lalana Kagal, editors, *The Semantic Web – ISWC 2020*, volume 12506, pages 56–73. Springer International Publishing, Cham, 2020.
3. Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, Jose Emilio Labra Gayo, Roberto Navigli, and Sebastian Neumaier. *Knowledge Graphs*. Springer, Cham, 1st edition edition, November 2021.
4. Maximilian Nickel, Kevin Murphy, Volker Tresp, and Evgeniy Gabrilovich. A Review of Relational Machine Learning for Knowledge Graphs. *Proceedings of the IEEE*, 104(1):11–33, January 2016.
5. Sebastian Rudolph. Foundations of Description Logics. In Axel Polleres, Claudia d’Amato, Marcelo Arenas, Siegfried Handschuh, Paula Kroner, Sascha Ossowski, and Peter Patel-Schneider, editors, *Reasoning Web. Semantic Technologies for the Web of Data: 7th International Summer School 2011, Galway, Ireland, August 23-27, 2011, Tutorial Lectures*, pages 76–136. Springer, Berlin, Heidelberg, 2011.
6. Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach, Global Edition: A Modern Approach, Global Edition*. Pearson, Boston, 4 edition, May 2021.
7. Taisuke Sato. A linear algebraic approach to datalog evaluation. *Theory and Practice of Logic Programming*, 17(3):244–265, May 2017.
8. Efthimis Tsilonis, Alexander Artikis, and Georgios Paliouras. A Tensor-Based Formalization of the Event Calculus. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, pages 3584–3592, Jeju, South Korea, August 2024. International Joint Conferences on Artificial Intelligence Organization.